READING GUIDE

Mathematics and decision rules

A reference to the BEMIX Policy-Aware 1.3.0 implementation, prepared for technical and program review. The original score, the policy severity and the financial award remain separate decisions.

The calculation in one line

$$P_{\text{calc}} = \min\left(C_g, \max_{k \in K} p_k\right) \qquad \text{M01}$$

Each validated impact produces an eligible financial candidate. The engine selects the largest applicable candidate and respects the published report-wide ceiling. It does not average a strong impact down because another weaker impact is also demonstrated.

Decision	What it answers
Original BEMIX	What do the four original inputs score?
Program severity	Which published impact rule fits the demonstrated behaviour?
Financial placement	Where does the finding sit inside that severity's financial band?
Final Award	What amount does the program confirm, including a permitted adjustment?

How to read this reference

Pages	Subject
02-04	Original score, classification and contextual criteria
05-09	Bounded importance, financial curves, caps and worked examples
10-13	Rounding, unknowns, configuration checks and review
14-18	Service mathematics, limits, notation and source coverage

Examples use synthetic settings. Importance ranges, the influence coefficient, weights and financial curves require calibration and platform approval before real use. This document explains the implementation; it does not approve a real bounty.

01 / ORIGINAL MODEL

The BEMIX score stays intact

$$E, D, O \in \{0,1,2,3,4\}, \qquad X \in \{0,1,2\} \tag{M02}$$

$$A = \max(E,D,O), \quad C = \min(E,D,O), \quad B = E + D + O - A - C$$

$$S_B = 5A + 2B + C + X - 4 \tag{M03}$$

E, D and O measure economic, confidentiality and operational impact. X is exploitability. A, B and C are the three impact inputs sorted from largest to smallest. When E = D = O = 0, the core returns Informational with no numeric score; the estimate view shows zero. An accepted assessment with that vector is flagged for evidence review.

Original band	Points	Illustrative marginal rate
Low	1-7	30
Medium	8-14	40
High	15-21	550
Critical	22-30	5,060

$$E = 4,\ D = O = 0 \quad \Longrightarrow \quad S_B = 16 + X \leq 18 < 22 \tag{M04}$$

A maximum impact on one axis reaches at most 18. The program may still validate Critical under its own rulebook. The original numerical label remains visible alongside that separate classification.

Legacy payout, kept for comparison

$$B_{\text{legacy}}(S) = \sum_t \max(0, \min(S, u_t) - \ell_t + 1)\, r_t \tag{M05}$$

$$B_{\text{legacy}}(17) = 7(30) + 7(40) + 3(550) = 2{,}140$$

$$\text{LegacyBand}_t = \left[B_{\text{legacy}}(\ell_t), B_{\text{legacy}}(u_t) \right]$$

The marginal formula counts inclusive points in each band. The rates above reproduce the historical USDT comparison fixture; they are not the policy-aware curve. A missing numeric score is treated as zero by this baseline function. Its band endpoints are the payouts at the first and last score of each tier.

Sources: bemix_pilot/engine.py · bemix() · bemix_pilot/baseline.py · baseline_bounty(), baseline_bands()

02 / CLASSIFICATION

Severity comes from applicable rules

Researchers declare behaviours. The triager validates or corrects the facts. A selected name is not, by itself, proof of severity.

$$M(r,f)=\begin{cases}0 & \text{a known fact contradicts a condition,}\\ ? & \text{no contradiction, but a required fact is unknown,}\\ 1 & \text{every required condition matches.}\end{cases} \quad \text{M06}$$

A known contradiction makes a rule false even if another condition is unknown. For an unless_all cap, a known match is inverted; unknown remains unknown.

$$M_{\text{unless_all}}=\begin{cases}1-M & M\in\{0,1\},\\ ? & M=?.\end{cases}$$

Apply precedence before selecting severity

$$T=\{r : M(r,f)=1\},\qquad T_{\text{eff}}=T\setminus D(T) \quad \text{M07}$$

$$s=\max_{r\in T_{\text{eff}}}\text{severity}(r) \quad \text{M08}$$

D(T) contains descendants overridden by matched rules in the same impact group. The precedence graph must be acyclic. If effective rules in one group retain different severities without precedence, the result stays a policy conflict. The ordering is Low < Medium < High < Critical.

$$K=\{k : \text{required}(k)\subseteq T_{\text{eff}},\ M(k.\text{when},f)=1\} \quad \text{M09}$$

An eligible profile needs all its required rules and its own conditions to match. The highest resolved severity also needs a financial profile of that severity. Empty, unmapped or materially unresolved cases do not receive a fabricated classification or price.

Source: bemix_pilot/engine.py · matches(), cap_state(), dominance(), _evaluate()

03 / CONTEXT

Measure the evidence within a profile

$$Q_k = \frac{\sum_{j \in J_k} w_{kj} q_{kj}}{\sum_{j \in J_k} w_{kj}} \tag{M10}$$

$$0 \le q_{kj} \le 1, \quad w_{kj} \ge 0, \quad \sum_j w_{kj} > 0 \quad \Longrightarrow \quad 0 \le Q_k \le 1$$

The fixed criterion set J belongs to the approved profile. A value q is a rubric level between zero and one, not automatically a success probability. Each w is a relative weight. Researchers cannot remove a low criterion from the denominator.

$$h_{kj} = \frac{w_{kj}}{\sum_\ell w_{k\ell}}, \qquad \sum_j h_{kj} = 1 \tag{M11}$$

Business risk is contextual

Financial reach, reliability, coverage, interaction and effective mitigations can contribute where relevant. Reputation and compliance can be included when applicable. A fund-loss impact does not need a separate KYC breach to qualify. Mandatory top-of-band conditions remain explicit eligibility or cap rules.

Optional mappings reuse original inputs

$$q_E = \frac{E}{4}, \quad q_D = \frac{D}{4}, \quad q_O = \frac{O}{4}, \quad q_X = \frac{X}{2} \tag{M12}$$

A platform-approved binding can derive an individual criterion from E, D, O or X. It is applied identically to researcher estimates and staff assessments. The full aggregate $S_B/30$ is not automatically blended back into every reward.

$$\frac{\partial Q_k}{\partial q_{kj}} = h_{kj} \ge 0, \qquad Q_k(\lambda w, q) = Q_k(w, q) \quad (\lambda > 0) \tag{D01}$$

For fixed weights, better criterion values cannot lower Q. Scaling all weights equally leaves their average unchanged; it does not give an impact greater importance. The interval checks still apply to any proposed weights.

Sources: engine.py · _evaluate() · bemix_service/estimates.py · apply_bindings() · guardrails.py · validate_configuration()

04 / PROGRAM PRIORITIES

Importance has a permitted range

The platform sets the intervals. The program chooses the relative importance of each impact inside its already validated severity.

$$0 < a_{\mathrm{L}} < b_{\mathrm{L}} < a_{\mathrm{M}} < b_{\mathrm{M}} < a_{\mathrm{H}} < b_{\mathrm{H}} < a_{\mathrm{C}} < b_{\mathrm{C}} \tag{M13}$$

$$I_k \in \mathbb{Z}, \quad a_s \le I_k \le b_s, \qquad i_k = \frac{I_k - a_s}{b_s - a_s} \in [0,1] \tag{M14}$$

Severity	Synthetic integer interval
Low	1 to 2
Medium	3 to 4
High	5 to 7
Critical	8 to 10

$$I = 8 \Rightarrow i = 0, \qquad I = 9 \Rightarrow i = \frac{1}{2}, \qquad I = 10 \Rightarrow i = 1$$

Normalized i = 0 means the minimum permitted importance in that category. For the Critical example it means I = 8, not zero impact. A High importance cannot be used as Critical, and a Critical profile cannot be assigned a Low value.

What the number controls

Importance affects part of the financial position inside the category. It is not added to the BEMIX score and cannot change High into Critical. Two equally important impact types may use the same value.

The scale is an experimental fixture. A hypothetical ranking of RCE, key disclosure, silent signing and withdrawal manipulation is a configuration choice to review, not a claim that one is universally worse than another.

05 / FINANCIAL POSITION

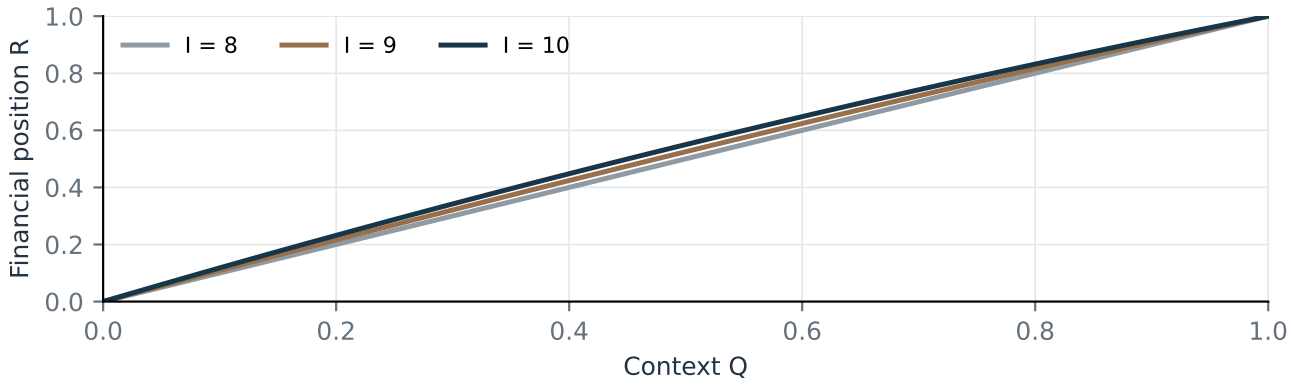
Add importance without a new ceiling

$$R_k = Q_k + \alpha\, i_k Q_k(1 - Q_k) \qquad \text{M15}$$

$$R(0, i) = 0, \qquad R(1, i) = 1, \qquad 0 \le R \le 1$$

The platform template fixes 0 < alpha ≤ 1. The generic engine also accepts zero. The example uses alpha = 1/5. Importance can raise intermediate placement while preserving both endpoints of the financial band.

$$\frac{\partial R}{\partial i} = \alpha Q(1 - Q) \ge 0, \qquad \frac{\partial R}{\partial Q} = 1 + \alpha i(1 - 2Q) \ge 0 \qquad \text{D02}$$



$$0 \le R - Q \le \frac{\alpha}{4}; \qquad \alpha = \frac{1}{5} \Rightarrow R - Q \le 0.05 \qquad \text{D03}$$

The maximum extra position is alpha/4. With alpha = 1/5, it is five percentage points of position, not a 20% bounty bonus. These derivatives describe a fixed profile; they do not prove that arbitrary changes to rule precedence are monotone.

Source: engine.py · reward_position() · D02-D03 are explanatory mathematical consequences of that function.

06 / CONVERSION TO MONEY

Use the approved financial curve

$$F_k(R) = L_s + (U_s - L_s)R \quad \text{M16}$$

L and U are the configured lower and upper rewards of the policy severity. The curve belongs to one program snapshot, asset and currency. The engine does not compare USDC and USDT as interchangeable values.

A curve may have intermediate anchors

$$0 = z_0 < z_1 < \cdots < z_m = 1, \quad L_s = p_0 \le p_1 \le \cdots \le p_m = U_s \quad \text{M17}$$

$$F_k(R) = p_h + \frac{p_{h+1} - p_h}{z_{h+1} - z_h}(R - z_h), \qquad z_h \le R \le z_{h+1} \quad \text{M18}$$

Position R	Illustrative anchor amount
0.00	4,500
0.25	7,500
0.50	13,500
0.75	27,500
1.00	50,000

At R = 0.35, these anchors produce 9,900 by straight-line interpolation. They are a different illustrative curve from the linear example; the engine does not mix the two.

Worked linear example

$$Q = 0.70,\ i = \frac{1}{2},\ \alpha = \frac{1}{5} \Rightarrow R = 0.721 \Rightarrow F(R) = 37{,}305.50 \quad \text{E01}$$

Here i = (9 - 8)/(10 - 8) = 1/2. The linear curve adds 4,500 to 45,500 × 0.721. Monotonicity makes the calculation reproducible; it does not establish that the selected curve or priorities are economically fair.

Sources: engine.py · interpolate(), validate_policy() · tests/test_transport_and_boundaries.py · piecewise-anchor example

07 / SELECTION AND RESTRICTIONS

Take the largest eligible candidate

$$C_k = \min\left(\{U_{s(k)}\} \cup \{c : \ c \text{ is an active cap on } k\}\right) \quad \text{M19}$$

$$p_k = \min(C_k, F_k(R_k)) \quad \text{M20}$$

$$C_g = \min(\{U_s\} \cup \{c : \ c \text{ is an active report-wide cap for } s\}) \quad \text{M21}$$

$$P_{\text{calc}} = \min\left(C_g, \max_{k \in K} p_k\right) \quad \text{M01}$$

Caps are applied only when their conditions and scope match. The band maximum supplies a ceiling when no cap applies. An unquantified relevant cap remains unresolved; it does not become a guessed discount.

Candidate	Illustrative amount
Main impact	31,800
Additional weaker impact	22,700
Maximum without a combined profile	31,800
Optional approved combined-impact profile	36,000

An additional candidate cannot reduce an existing one while that candidate remains eligible. Payments are not automatically summed. A change in rule precedence can remove a candidate; that is a separate revision issue.

$$E_{\text{award}} = \min\left(C_g, \max_{k \in K} C_k\right) \quad \text{M22}$$

The service also calculates this eligible ceiling for later award adjustments. It is the minimum of the report-wide ceiling and the largest eligible profile ceiling. It is not necessarily the ceiling of only the profile that won on its current evidence.

Source: engine.py · _evaluate() · profiles, global_cap_rules, eligible_ceiling_minor_units

08 / WORKED EXAMPLE

One case, three recorded amounts

A synthetic Critical profile uses the 4,500-50,000 USDC band, equal context weights and minimum permitted importance I = 8. The original BEMIX inputs remain E = 4, D = O = 0 and X = 1: score 17.

Researcher declaration

$$Q = \frac{1(0.60) + 1(0.80)}{1+1} = 0.70, \quad i = 0, \quad R = 0.70 \tag{E02}$$

$$4{,}500 + (50{,}000 - 4{,}500)(0.70) = 36{,}350$$

The engine can calculate this estimate before the submitted facts are validated. An exact amount is not proof that the declared inputs are correct.

Triager correction

$$Q = \frac{0.60 + 0.60}{2} = 0.60 \quad \Rightarrow \quad P_{\text{calc}} = 31{,}800 \tag{E03}$$

Reliability changes from 0.80 to 0.60. The estimate remains in history and the validated amount is recalculated. A justified correction may raise or lower the value.

Program-confirmed award

$$P_{\text{award}} = 31{,}800 + 500 = 32{,}300 \tag{E04}$$

The permitted +500 adjustment is recorded separately from the calculated bounty. It does not manufacture BEMIX points and does not execute a transfer.

Same context Q = 0.60	Position R	Linear bounty
Importance 8	0.600	31,800
Importance 9	0.624	32,892
Importance 10	0.648	33,984

Sources: bemix_pilot/lifecycle_example.py · demonstration() · platform_example.py · importance_demonstration()

09 / PRECISION AND AWARD

Round once, then record the adjustment

Calculations use exact fractions. Monetary inputs and raw stored amounts use integer minor units; display amounts are decimal strings. This release fixes 100 minor units per unit of the reward currency. These are not native token units.

$$\rho(x) = \left\lfloor x + \frac{1}{2} \right\rfloor = \left\lfloor \frac{2n + d}{2d} \right\rfloor, \qquad x = \frac{n}{d} \geq 0 \tag{M23}$$

$$B = \min\left(\rho(P_{\text{calc}}), C_g, U_s\right), \qquad \text{display}(B) = \frac{B}{100} \tag{M24}$$

The formula is for non-negative raw payouts measured in minor units. For example, 450,000.5 minor units rounds to 450,001, displayed as 4,500.01. The engine also bounds the rounded result by the applicable integer cap and band maximum.

$$P_{\text{award}} = B + \delta, \qquad \delta_{\min} \leq \delta \leq \delta_{\max} \tag{M25}$$

$$A_{\min} \leq B + \delta \leq A_{\max}, \qquad [A_{\min}, A_{\max}] = [L_s, E_{\text{award}}] \tag{M26}$$

Normally the final award stays between the severity floor and eligible ceiling. A published explicit exception may replace those two award bounds. An adjustment needs its policy rule and a recorded reason; a signed delta is stored as integer minor units.

$$\delta \in [\delta_{\min}, \delta_{\max}] \cap [A_{\min} - B, A_{\max} - B] \tag{D04}$$

This interval intersection is a useful restatement of the two checks in the code. Without an adjustment, delta is zero. An ineligible finding can receive a confirmed zero award, with no adjustment. Materially unresolved calculations cannot be finalized.

Sources: engine.py · round_minor(), money(), _evaluate() · bemix_service/awards.py · calculate_award()

10 / INCOMPLETE INPUTS

Unknown is different from unverified

Complete declared values can support an exact provisional estimate. When numerical values or boolean conditions are genuinely missing, the engine asks whether they can change the relevant result. It never promotes hypothetical completions to observed facts.

$$N_{\text{boolean}} = 2^n \le 256, \qquad N_{\text{attempts}} = 2 \cdot 2^n \le 512 \quad (n \le 8) \tag{M27}$$

For each boolean completion, missing positive-weight metric values are jointly set to 0 and jointly set to 1. This uses monotonicity within a fixed profile; it does not enumerate every mixed combination of metric endpoints. More than eight unresolved boolean facts requests additional input.

$$P_{\min} = \min_{\omega \in \Omega} P(\omega), \qquad P_{\max} = \max_{\omega \in \Omega} P(\omega) \tag{M28}$$

$$P_{\min} = P_{\max} \quad \text{and} \quad \left|\{s(\omega) : \omega \in \Omega\}\right| = 1 \tag{M29}$$

An exact result requires every attempted branch to calculate successfully, one common severity and equality of the unrounded bounds. If any branch stays unresolved, the result stays conditional. A range can be exposed when the supported branches are all calculable but differ.

$$T_{\text{certain}} = \bigcap_{\omega \in \Omega} T_{\text{eff}}(\omega), \quad W_{\text{certain}} = \bigcap_{\omega \in \Omega} W(\omega) \tag{M30}$$

$$E_{\text{safe}} = \min_{\omega \in \Omega} E_{\text{award}}(\omega) \tag{M31}$$

For an exact result despite unknowns, the engine retains rules and winning profiles common to all completions. It takes the smallest eligible award ceiling across them. This avoids presenting an adjustment allowance that depends on an unresolved favourable assumption.

Source: engine.py · evaluate() · conditional completions, intersections and safe eligible ceiling

11 / PLATFORM CONFIGURATION

Bounds apply to weights and shares

$$w_{kj}^{\min} \le w_{kj} \le w_{kj}^{\max}, \qquad h_{kj}^{\min} \le h_{kj} \le h_{kj}^{\max} \tag{M32}$$

$$\sum_j h_{kj}^{\min} \le 1 \le \sum_j h_{kj}^{\max}, \qquad 0 \le h_{kj}^{\min} \le h_{kj}^{\max} \le 1 \tag{M33}$$

A valid weight alone is not enough: its share of the total must also stay inside the allowed interval. Applicable criteria require positive weight minima. Only a criterion fixed to zero by the platform may be disabled. Rubrics, mappings, curves and caps are protected template structure.

Run the configuration through the engine

$$Q_t = \frac{t}{20}\ (t = 0, \ldots, 20), \quad N_{\text{grid}} = 21 \sum_{k \in K_{\text{config}}} (b_{s(k)} - a_{s(k)} + 1) \tag{M34}$$

$$N_{\text{transition}} \le \sum_{k \in K_{\text{config}}} (3|J_k| + 18) \tag{M35}$$

For every allowed integer importance and 21 context levels, the validator checks the band, non-decreasing money and both endpoints. It also tests each criterion at 0, 1/2 and 1, plus every integer BEMIX value in a full-engine acceptance scenario for each profile.

Supplied synthetic template	Recomputed count
Approved impact profiles	7
Financial grid points	420
BEMIX / criterion transition checks	161

The transition bound uses 3 levels per criterion and at most 18 BEMIX values per profile. All-zero impact combinations are skipped. The finite checks complement the continuous monotonicity argument; neither proves fair calibration or all possible precedence transitions.

Sources: bemix_service/guardrails.py · validate_template(), validate_configuration(), certify_configuration()

12 / REVIEW AND REVISIONS

Approval of rules is separate from triage

$$\text{Live}(p) \Rightarrow \text{Valid}(p,t) \wedge \text{TestsPass}(p,t) \wedge \text{PlatformApproves}(H_p, H_t) \quad \text{M36}$$

The program requests publication. The platform reviews the exact policy record, template and validation certificate. Only platform approval activates it. A proposal can be returned for changes even if its arithmetic passes. New versions require new approval; submitted reports retain their snapshots.

An improvement claim has a stricter contract

$$V_0 \subseteq V_1, \quad q_{kj}^{(1)} \geq q_{kj}^{(0)}, \quad C_k^{(1)} \geq C_k^{(0)} \quad \text{M37}$$

$$P_1 \geq P_0 \quad \text{for a supported improvement under the same policy} \quad \text{M38}$$

V denotes the calculated profile candidates retained in each assessment. The checker also rejects decreased BEMIX axes, changed eligibility, lost positive-weight evidence, tightened ceilings, reduced severity or fact changes outside the policy's favourable directions. Comparisons use the same frozen policy.

$$40{,}900 \longrightarrow 22{,}700 \quad \text{when precedence removes an earlier profile} \quad \text{E05}$$

The supplied precedence example removes an earlier profile despite wider coverage. A client calling that an improvement is asked to record a justified correction. The assigned triager can correct during triage; after handoff, program security can reassess. Another triager is not required.

Revision checks compare exact raw money when both decisions are calculated. A pending proposal stays pending. A later assessment makes an older award stale until it is reconfirmed; history is preserved.

Sources: service.py · approve_policy(), _publish_policy() · bemix_pilot/revisions.py · evaluate_revision()

APPENDIX A / INTEGRITY

Hashes identify the reviewed objects

These equations support identity, reproducibility and the audit trail. They are not reward weights or additional severity factors.

$$H(x) = \mathrm{SHA256}(\mathrm{UTF8}(J(\mathrm{canonical}(x)))) \tag{S01}$$

The core canonicalizer converts dataclasses, sequences and exact fractions into stable JSON-compatible values. J uses sorted keys, compact separators and UTF-8. Hashes identify the precise policy and numerical inputs; researcher display labels are excluded from the scoring input fingerprint.

$$h_0 = 0^{64}, \quad h_n = \mathrm{SHA256}(\mathrm{UTF8}(J(e_n))), \quad e_n.\mathrm{previous_hash} = h_{n-1} \tag{S02}$$

$$\mathrm{MAC}_d(h) = \mathrm{HMAC}_{\mathrm{SHA256}}(K, \mathrm{UTF8}(d \| : \| \mathrm{store_id} \| : \| h)) \tag{S03}$$

An event hash includes its previous hash and sequence-bearing fields. Each stored record/event is authenticated with a domain such as event, record, idempotency or checkpoint. The MAC message uses the literal colon separators and hexadecimal digest text shown above.

$$d_{\mathrm{req}} = \mathrm{SHA256}(\mathrm{UTF8}(J(\{\mathrm{operation}, \mathrm{resource}, \mathrm{body}\}))) \tag{S04}$$

The request digest binds operation, resource and body. Reuse of an idempotency key for the same program and subject returns the previous response only when the operation and digest agree. The stored response record has its own authenticated hash.

Storage hashes use its strict JSON encoder on already prepared fields. Record bodies and event payloads are serialized JSON strings inside the hashed envelopes, not silently substituted nested objects. HMAC is shared-key authentication, not public proof against a host administrator who controls the key.

Sources: engine.py · canonical(), fingerprint() · storage.py · encode(), event(), insert(), _mac() · service.py · command()

APPENDIX B / SERVICE CONTROLS

Time, identity and version checks

$$d_{\text{token}} = \text{SHA256}(\text{UTF8}(\text{token})) \tag{S05}$$

$$t_{\text{expiry}} = t_{\text{issued}} + d\,\text{days}, \quad 1 \le d \le 90, \quad t_{\text{now}} < t_{\text{expiry}} \tag{S06}$$

A token contains 32 random bytes encoded in URL-safe form with a bmx_ prefix. The digest is stored instead of the bearer value. Acceptance also requires an active principal, current membership and a token that is neither revoked nor expired.

$$W(t) = \left\lfloor \frac{t}{60} \right\rfloor, \qquad c_t = \begin{cases} c_{\text{prev}} + 1 & W(t) = W_{\text{prev}}, \\ 1 & \text{otherwise.} \end{cases} \tag{S07}$$

$$c_t > r_{\text{minute}} \quad \Rightarrow \quad \text{HTTP 429} \tag{S08}$$

The default service allowance is 120 requests per minute per authenticated subject. The counter is persisted and an over-limit request receives HTTP 429. Health endpoints do not use this authenticated request counter.

$$v_{\text{expected}} = v_{\text{stored}}, \qquad \text{awardCurrent} \iff H_{\text{award.assessment}} = H_{\text{latest assessment}} \tag{S09}$$

Updates must identify the current version. A Final Award is current only while it points to the latest assessment hash. For audit rollback detection, the authenticated external checkpoint must refer to the same store, with its sequence/hash still present in the database history.

$$\text{elapsed}_{\text{ms}} = 1000 \left(t_{\text{end}}^{\text{monotonic}} - t_{\text{start}}^{\text{monotonic}} \right) \tag{S10}$$

This logged elapsed time uses a monotonic clock. It is an operational measurement, unrelated to exploitability or reward placement.

Sources: service.py · issue_token(), _identity(), _consume_rate(), _expected(), read_case() · storage.py · verify() · api.py

APPENDIX C / DOMAINS

Implementation limits are not payout factors

$$0 \le L_s < U_s, \qquad L_{s+1} \ge U_s \tag{C01}$$

$$|\mathcal{R}|, |\mathcal{P}|, |\mathcal{C}| \le 256, \quad |J_k| \le 64, \quad 2 \le |\mathcal{A}_k| \le 64 \tag{C02}$$

$$b_s - a_s \le 100 \quad \Rightarrow \quad \#\text{integer choices} \le 101 \tag{C03}$$

Constraint	Implemented bound
Context values / normalized shares	0 to 1; exact fractions
Importance influence	0 to 1 in core; strictly positive in a platform template
Submission catalog / evidence prompts	At most 64 options; at most 8 prompts per option
Behaviours in one declaration	1 to 8; identifiers cannot repeat
Published adjustment rules	At most 32
JSON document / HTTP body	2,000,000 bytes / 1,048,576 bytes
Idempotency key / token validity	16-128 characters / 1-90 days
History page / offset	1-50 entries / 0-1,000,000
HTTP request-body read	10-second timeout
Database / operator audit key	At least 32 key bytes; API key file exactly 32 bytes

Published bands are ordered and may touch at a boundary. A display amount at a shared boundary does not change its validated severity. Identifier/text limits, version strings, dependency hashes and test timeouts are transport or operational configuration, not new scoring equations.

Sources: engine.py · serialization.py · submissions.py · awards.py · guardrails.py · service.py · api.py · storage.py

APPENDIX D / NOTATION

A compact symbol reference

Symbol	Meaning
E, D, O, X	Original economic, data, operational and exploitability inputs
A, B, C	E, D and O sorted from highest to lowest
S_B	Original BEMIX score; separate from program severity
s; k; j	Severity; impact profile; criterion in that profile
T; D(T); T_{eff}	Matched rules; overridden descendants; effective rules
K; J_k	Eligible impact profiles; fixed criteria of profile k
q_{kj}; w_{kj}; h_{kj}	Criterion level; relative weight; normalized weight share
Q_k	Contextual weighted average
I_k; $[a_s, b_s]$	Chosen integer importance; platform severity interval
i_k; alpha	Normalized importance; platform influence coefficient
R_k	Financial position after bounded importance adjustment
L_s; U_s	Financial band floor and maximum
(z_h, p_h)	Position and monetary value of a curve anchor
F_k; p_k	Financial curve; capped candidate for profile k
C_k; C_g	Effective profile cap; effective report-wide cap
P_{calc}; B	Raw calculated payout; rounded integer minor-unit bounty
E_{award}; delta	Eligible ceiling for adjustments; separate signed adjustment
P_{award}	Program-confirmed amount; not an executed payment
Omega; W(omega)	Calculated conditional completions; winning profiles in one completion
H; J; K in Appendix A	Hash function/digest; stable JSON encoding; secret HMAC key

Monetary formulas are scale-consistent: engine calculations use minor units throughout; display examples use currency units throughout. Convert by 100 only at the display boundary. A letter used for a cryptographic key in the appendix is not the set of financial profiles.

Notation follows the supplied source. Curve coordinates use z to avoid confusing them with importance-interval endpoints a and b.

APPENDIX E / SOURCE COVERAGE

What was checked against the repository

Source / functions	Equations or subject
bemix_pilot/engine.py bemix, matches, dominance	M02-M04, M06-M09: score, rule matching and severity
engine.py reward_position, interpolate, _evaluate	M10-M24: context, importance, curves, caps and rounding
engine.py evaluate	M27-M31: conditional bounds and conservative award ceiling
bemix_pilot/baseline.py	M05 and legacy band endpoints
bemix_service/estimates.py	M12: mappings; estimate/assessment arithmetic
bemix_service/awards.py	M25-M26, D04: final amount and adjustment constraints
bemix_service/guardrails.py	M32-M35: raw weights, shares and certification grid
bemix_pilot/revisions.py bemix_service/service.py	M36-M38: revisions, exact-hash platform approval
engine.py · storage.py · service.py	S01-S09: fingerprints, authenticated history, identity and versions
api.py · serialization.py · submissions.py	S10, C01-C03 and transport/cardinality limits
examples.py · lifecycle_example.py platform_example.py · revision_demo.py	Worked examples E01-E05; synthetic parameters

The audit inventory covers 77 text files, including 7 Markdown documents. Tests, fixtures, generated JSON, scripts, deployment files and dependency locks were read/scanned for additional relationships. Test vectors and repeated serialized formulas are not new independent payout rules.

The earlier expanded PDF already covers the central model. This edition adds explicit eligible-adjustment ceilings, safe ceilings across unknown completions, weight-scaling and sensitivity explanations, and a clearer division between executable rules, derived properties and examples.

Source snapshot SHA-256: f9be91406f7d87566e7785a04d950066a10945cd678f242824fb13027d5baf50

M = implemented relationship; D = explanatory derivation; E = numerical example; S = supporting service calculation; C = implementation constraint. Snapshot-based review, not a claim about a later GitHub commit. The supplied verification records 215 tests; this document independently recomputed its displayed numerical examples and configuration counts.